DOCUMENTO DE ARQUITECTURA E INTEGRACIÓN TÉCNICA

Solución Caso Práctico: TongoMart (Uganda)

Diseño de Integración End-to-End entre E-Commerce (Magento), Facturación Automatizada con Sistema POS y La Plataforma Logística (Last-Mile Delivery Engine - LMD).

CLIENTE & GEOGRAFÍA
TongoMart Supermarkets — Kampala, Uganda (UGX)

ECOSISTEMA TECNOLÓGICO
Magento OMS + Middleware + POS + LMD API

MODELO OPERATIVO
Full Service & Pick and Delivery (Pickers + Drivers)

ROL TÉCNICO
Senior Solutions Engineer / ADSO

CONTENIDO EJECUTIVO DEL ENTREGABLE

Punto 1: Mapeo Completo de Endpoints e Integración para TongoMart — Sección 1

Punto 2: Arquitectura de Componentes y Secuencia End-to-End (POS) — Sección 2

Punto 3: Automatización en Postman (Pre-request Script Dinámico y Flujo) — Sección 3

Punto 4: Auditoría Técnica del Payload JSON + Payload Corregido (Uganda) — Sección 4

Punto 5: Propuestas Técnicas de Mejora para la API (DX & Resiliencia) — Sección 5

Clasificación: Confidencial — Uso Exclusivo para Evaluación Técnica (Cumplimiento NDA)

Noviembre 2026

PUNTO 1 DEL CASO PRÁCTICO

1. Mapeo Completo de Endpoints e Integración para TongoMart

TongoMart es una cadena de supermercados en Uganda (con operación principal en Kampala) que cuenta con una tienda virtual desarrollada sobre **Magento** y busca digitalizar y orquestar su operación de última milla mediante **La Plataforma Logística (Last-Mile Delivery Engine - LMD)**.

Para cumplir con los requerimientos del cliente, la integración resuelve **4 pilares críticos de negocio**: (1) Integración nativa con su E-commerce en Magento, (2) Promesa de entrega precisa para el comprador final mediante cálculo de *Time Slots* en tiempo real, (3) Orquestación en tienda y ruta de *Pickers* y *Drivers*, y (4) Facturación 100% automatizada con su **Sistema POS** antes de finalizar el paso por caja.

1.1. Matriz General de Mapeo de Endpoints y Componentes

Módulo / Necesidad de TongoMart	Endpoint / Componente	Método y Ruta	Rol en la Integración
1. Sincronización de Catálogo y Stock (Magento → LMD)	Batch Products & Batch Catalog	`POST` / `PUT` Catalog Endpoints	Sincronizar SKUs, códigos de barras (`barcodes`), precios en `UGX` , dimensiones/pesos (para *Cartonization*) y ubicación en pasillos (`location` , `picking_index`) para guiar a los pickers en tienda.
2. Promesa de Entrega Precisa en Checkout	Availability (Time Slots) V2	`POST` `/jobs/availability/v2`	Consultar ventanas de entrega en tiempo real en el checkout de Magento según capacidad instalada de pickers, flota de drivers y tamaño del carrito.
3. Reserva y Extensión de Slot (Recomendado)	Check Slot / Extend Slot Expiration	`GET` `/jobs/{slotId}/check` `PUT` `/jobs/{slotId}/extend_expiration_time`	Validar que el slot elegido siga vigente mientras el cliente procesa el pago en Magento, evitando el error `417 Expectation Failed` .
4. Creación de la Orden para Pickers y Drivers	Create a Job	`POST` `/jobs`	Disparar la orden confirmada en Magento hacia la app de Pickers y Control Tower, enviando `payment.blocking_policy = "CHECKOUT"` para retener el paso de caja hasta que el POS facture.
5. Consulta Puntual de Estado (Backoffice)	Get Job by ID	`GET` `/jobs/{jobId}`	Obtener el detalle consolidado de un pedido para vistas de resumen y atención al cliente en Magento (sin hacer *polling* continuo).
6. Escucha de Eventos Operativos en Tiempo Real	Webhook 1: Job Tracking Statuses	`POST` `https://api.tongomart.ug/webhooks/logistics`	Recibir eventos firmados con `SHA-256` . El evento crítico es `PICKING_FINISHED` , que entrega los ítems realmente recolectados (`found_quantity`), adiciones y sustituciones hechas por el picker.

Módulo / Necesidad de TongoMart	Endpoint / Componente	Método y Ruta	Rol en la Integración
7. Facturación Automatizada con el POS	Payment Info + *Automatic Shopper App Checkout*	`PUT` `/jobs/{jobId}/payment_info`	Una vez que el POS de TongoMart emite la factura con los ítems reales de `PICKING_FINISHED`, actualiza en la plataforma el monto final (`prices`), adjunta la factura (`invoice`) y envía `blocking_policy: "UNBLOCKED"` .
8. Seguimiento en Vivo para el Cliente	Live Status Widget V3.x	Componente Frontend Web/Móvil	Mostrar al comprador en Magento el progreso del pedido y mapa del conductor en tiempo real, incluyendo el código de seguridad `delivery_code` .

1.2. Detalle Técnico por Fase del Ciclo de Vida del Pedido

A. Sincronización Previa de Catálogo y Cartonización

Magento alimenta el catálogo con `barcodes` , peso (`weight` en kg) y dimensiones espaciales oficiales (`x` , `y` , `z`) para que el motor de *Cartonization* asigne el vehículo adecuado (moto vs. van en Kampala) y ordene la ruta del picker según `location` y `picking_index` .

B. Consulta de Disponibilidad (POST /jobs/availability/v2)

Al ingresar la dirección en Magento, se geocodifica a coordenadas (`latitude` , `longitude`) y se evalúa la capacidad instalada de la tienda (`store_reference: "1234"`) bajo los modelos `["FULL_SERVICE", "PICK_AND_DELIVERY"]` y moneda `"UGX"` .

C. Creación con Bloqueo de Caja (POST /jobs)

Al confirmar la compra en Magento, se consume el `slot_id` reservado configurando `payment.blocking_policy = "CHECKOUT"` . Esto permite que el picker recolecte los artículos pero **retiene el paso de caja** hasta recibir la factura fiscal emitida por el POS.

D. Facturación Automática POS tras PICKING_FINISHED

Al finalizar el alistamiento, el Webhook `PICKING_FINISHED` notifica al Middleware los ítems reales (`found_quantity`). El POS factura el monto exacto y el Middleware invoca `PUT /jobs/{jobId}/payment_info` con `blocking_policy: "UNBLOCKED"` y el PDF fiscal.

Habilitación de Automatic Shopper App Checkout

Combinando `PUT /jobs/{jobId}/payment_info` (`blocking_policy: "UNBLOCKED"` + objeto `invoice`) con el ajuste operativo **Automatic Shopper App Checkout** en la plataforma, la orden supera automáticamente la etapa de checkout en la aplicación móvil del picker sin requerir escaneo manual en una caja registradora física, liberando de inmediato el pedido al conductor (*Driver*).

PUNTO 2 DEL CASO PRÁCTICO

2. Arquitectura de Integración y Flujos de Secuencia

La arquitectura diseñada para **TongoMart (Uganda)** desacopla el e-commerce en **Magento**, el **Sistema POS** y **La Plataforma Logística (LMD)** mediante un **Middleware / Orquestador de Integración** orientado a eventos (*Event-Driven Architecture*).

2.1. Diagrama 1: Arquitectura de Componentes en 5 Capas

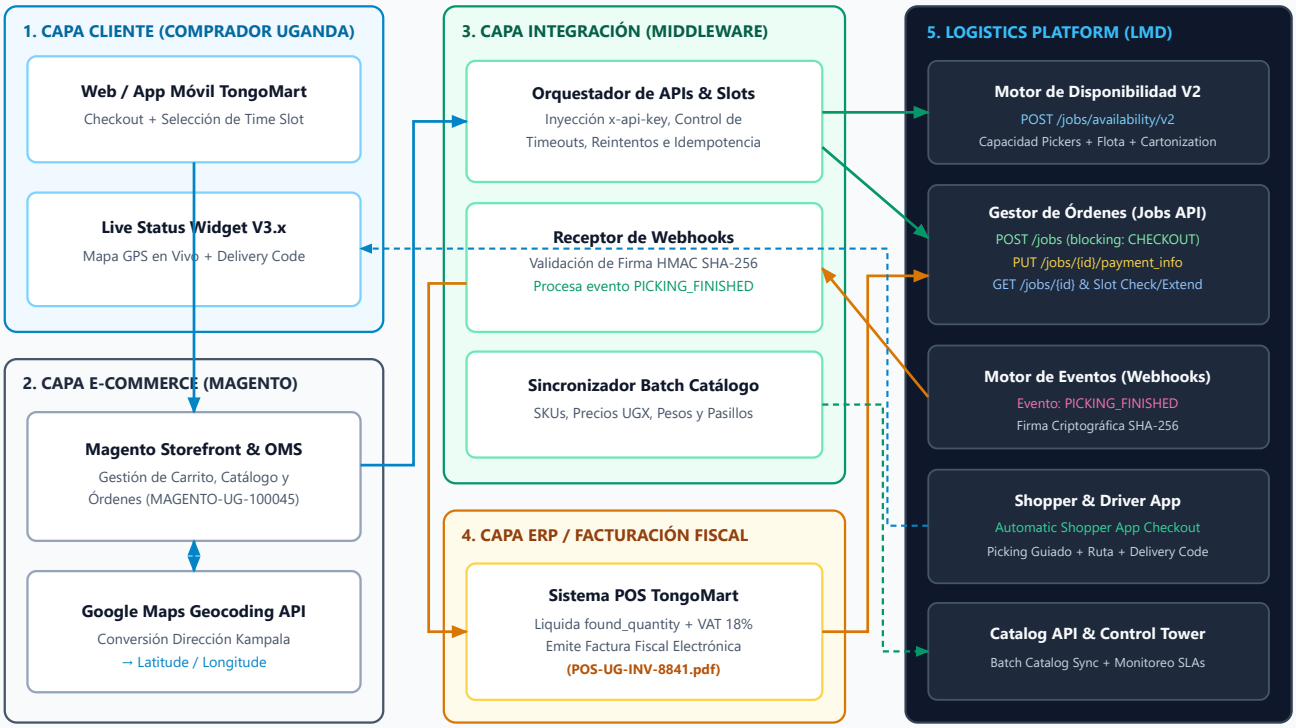


Figura 2.1 — Arquitectura de Integración Desacoplada en 5 Capas para TongoMart (Kampala, Uganda)

2.2. Diagrama 2: Flujo de Secuencia End-to-End con Facturación Automática POS

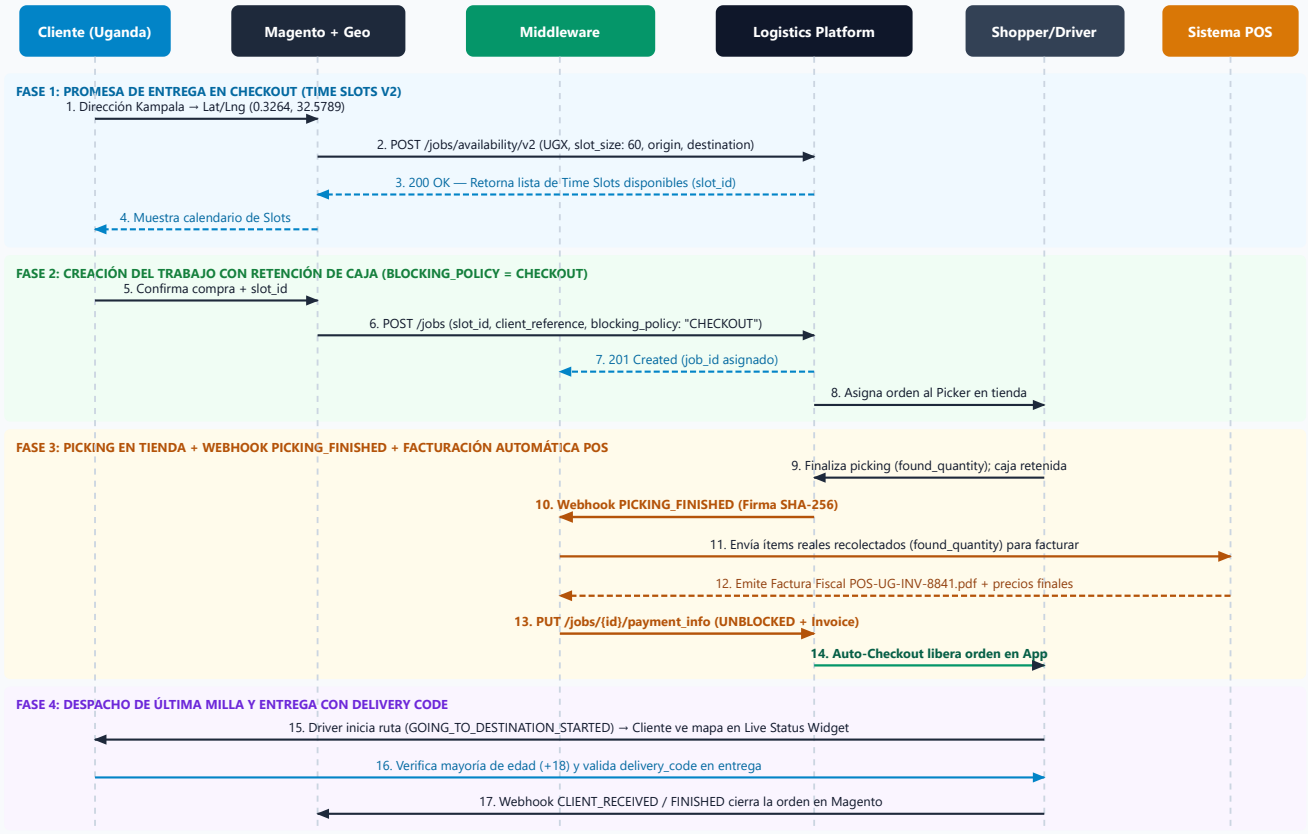


Figura 2.2 — Diagrama de Secuencia End-to-End incluyendo Bloqueo de Checkout y Facturación Automática con el POS

PUNTO 3 DEL CASO PRÁCTICO

3. Colección de Postman y Automatización de Variables

Se construyeron los archivos oficiales `postman/TongoMart_Logistics.postman_collection.json` (formato Postman v2.1.0) y `postman/TongoMart_Environment.postman_environment.json` listos para importar y ejecutar.

3.1. Variables del Environment Configuradas

Variable	Valor Inicial / Configuración	Descripción Técnica
`{{base_url}}`	URL Base del Servidor LMD	Parametrizada en el Environment para todos los requests de la colección.
`{{api_key}}`	`tm_live_987654321_uganda_pos_key`	Inyectada automáticamente en la cabecera HTTP `x-api-key` de cada petición.
`{{start}}` y `{{end}}`	Calculadas dinámicamente	Pobladas antes de cada ejecución mediante la función JavaScript `setDynamicAvailabilityDates()`.
`{{slot_id}}`	`slot_ug_kampala_001`	Se actualiza automáticamente con el `res[0].id` devuelto por `POST /jobs/availability/v2`.
`{{job_id}}`	`job_ug_kampala_999`	Se actualiza automáticamente con el `job_id` devuelto por `POST /jobs`.

3.2. Función JavaScript en el Pre-request Script (Cálculo Dinámico de Fechas)

El endpoint `POST /jobs/availability/v2` exige que `start` sea como máximo 5 minutos anterior a la hora actual del servidor y `end` sea posterior a `start` (máximo 7 días):

```
function setDynamicAvailabilityDates() {
    const now = new Date();
    const startIso = now.toISOString();
    // Sumar 24 horas (dentro del límite de 7 días posteriores a start)
    const endIso = new Date(now.getTime() + (24 * 60 * 60 * 1000)).toISOString();
    pm.environment.set("start", startIso);
    pm.environment.set("end", endIso);
}
setDynamicAvailabilityDates();
```

3.3. Estructura de Peticiones Incluidas en la Colección

1. **Carpeta 1 — Core Fulfilment Flow (Obligatorio Punto 3):**
   - `POST` **Availability (Time Slots) V2** (`{{base_url}}/jobs/availability/v2`): Evalúa slots en Kampala con moneda `UGX`, `slot_size: 60` y guarda automáticamente el `slot_id`.
   - `POST` **Create a Job** (`{{base_url}}/jobs`): Consume `{{slot_id}}`, configura `payment.blocking_policy = "CHECKOUT"` y captura automáticamente el `{{job_id}}`.
   - `GET` **Get a Job by ID** (`{{base_url}}/jobs/{{job_id}}`): Consulta el estado y trazabilidad de la orden creada.
2. **Carpeta 2 — Automated POS Billing & Slot Validation (Complementario Punto 1):**
   - `GET` **Check Slot Validity** (`{{base_url}}/jobs/{{slot_id}}/check`)
   - `PUT` **Extend Slot Expiration Time** (`{{base_url}}/jobs/{{slot_id}}/extend_expiration_time`)
   - `PUT` **Update Payment Info & Automated POS Invoice** (`{{base_url}}/jobs/{{job_id}}/payment_info`): Envía precios definitivos, adjunta la factura `POS-UG-INV-8841.pdf` y cambia `blocking_policy` a `"UNBLOCKED"`.

PUNTO 4 DEL CASO PRÁCTICO

4. Auditoría Técnica del Payload JSON (Availability V2)

Pregunta del Caso	Dictamen	Resumen Técnico del Hallazgo
4.1. ¿Es suficiente la información del JSON para obtener una respuesta del sistema?	NO ES SUFICIENTE	Rechazo inmediato con HTTP `400 Bad Request` / `422 Unprocessable Entity` por **6 errores bloqueantes de esquema y validación**.
4.2. ¿Es correcto el tipo de dato en los campos de acuerdo con la documentación?	INCORRECTO	Existen **3 violaciones directas de tipos de datos** (`store_reference` , `is_big_order` y `attributes.picking_index`).
4.3. ¿Algún comentario adicional que quisieras hacerle al cliente (TongoMart)?	5 MEJORAS CLAVE	Inconsistencia geográfica/monetaria (Colombia `COP` vs. Uganda `UGX`), modelo sin conductores, error de suma en `order_value` , pesos/dimensiones en cero y validación de edad (+18).

4.1. ¿Es suficiente la información del JSON para obtener respuesta? — No (6 Errores Bloqueantes)

1. **Omisión del parámetro obligatorio `slot_size` :** El endpoint `POST /jobs/availability/v2` exige en la raíz el atributo numérico `slot_size` (múltiplo de 15, ej. `60`) para definir la duración en minutos de las ventanas horarias a evaluar.
2. **Omisión del objeto obligatorio `origin` :** Además de `store_reference` , el contrato V2 exige el objeto `origin` con el nombre, dirección física oficial (`address` , `country` , `city` , `state`) y coordenadas (`latitude` , `longitude`) de la tienda de origen.
3. **Inversión cronológica en `start` y `end` y fechas en el pasado:** Se envía `"start": "2026-01-30T17:52:29.991Z"` y `"end": "2026-01-28T01:55:36.048Z"` . La fecha de fin (`end`) es **2 días anterior** a la fecha de inicio (`start`). Además, `start` no puede ser más de 5 minutos anterior a la hora actual del servidor y `end` debe ser máximo 7 días posterior a `start` .
4. **Valor inexistente en el enum `operational_models_priority` :** Se envía `"PICK_COLLECT"` , valor que no existe en el enumerador oficial. Los valores válidos son `FULL_SERVICE` , `PICK_AND_DELIVERY` , `PICK_AND_COLLECT` , `PICK_AND_DELIVERY_WITH_STORAGE` , entre otros.
5. **Producto con cantidad en cero (`quantity: 0`):** El ítem `"404670"` (*Tortilla De Harina*) tiene `"quantity": 0` y `"sub_quantity": 0` . Todos los productos enviados deben tener cantidad estrictamente mayor a cero (`> 0`) para calcular tiempos de alistamiento.
6. **Objeto `quantity_found_limits` vacío (`{}`):** Cuando se incluye este objeto (diseñado para productos pesables), requiere obligatoriamente las propiedades numéricas `min` y `max` (ej. `{"min": 1, "max": 5}`). Para productos por pieza (`"unit": "pz"`), debe omitirse.

4.2. ¿Es correcto el tipo de dato en los campos? — No (3 Violaciones de Tipo)

Campo en el Payload	Valor Original Enviado	Tipo Enviado	Tipo Exigido por la API	Valor Corregido
`store_reference`	`1234`	`number` (integer)	**`string`**	`"1234"`
`is_big_order`	`"false"`	`string`	**`boolean`**	`false`
`job_items[].attributes.picking_index`	`1` , `1` , `2`	`number` (integer)	**`string`**	`"1"` , `"1"` , `"2"`

4.3. Comentarios Consultivos Adicionales para el Cliente (TongoMart)

- **1. Coherencia Geográfica y Cambiaria (Uganda vs. Colombia/México):** TongoMart opera en **Uganda**, pero el JSON usa `"currency_code": "COP"` (Peso Colombiano), coordenadas de Bogotá, Colombia (`lat: 4.5321, lng: -74.1186` , `"Calle 50, 75 - 8"`) y deja vacíos los campos `country` , `city` y `state` . Deben configurar **`"currency_code":`**

`"UGX"` (Chelín Ugandés, ISO-4217), coordenadas reales dentro de los polígonos de cobertura de sus tiendas en Uganda (ej. Kampala: `lat: 0.3406, lng: 32.5862`) y la llave oficial `"address"`; de lo contrario, el motor devolverá cero slots disponibles por estar fuera de geocerca.

- **2. Modelo Operativo vs. Necesidad de Conductores (Drivers):** TongoMart solicitó organizar tanto a **pickers como a conductores (drivers)** para entregas a domicilio, pero en el payload intentaron configurar retiro en tienda (`PICK_AND_COLLECT`) y pusieron `"name": "CEDI 1"` como destino. Deben priorizar `["FULL_SERVICE", "PICK_AND_DELIVERY"]` para evaluar tanto la capacidad de picking en tienda como la flota de última milla.
- **3. Discrepancia Matemática en el Valor Total (`order_value`):** La sumatoria real de los productos en el JSON original es `(1 × 21.9) + (0 × 25.9) + (5 × 24.9) = 146.4` (o `172.3` corrigiendo la cantidad del segundo ítem a `1`), mientras que `payment_info.order_value` reporta `42.75`. Enviar un `order_value` exacto desde el checkout es indispensable para validar límites de efectivo (`CASH`) de los conductores.
- **4. Pesos y Volúmenes en Cero (`weight: 0`, `volume: 0` / `volume: 1` genérico) y Cartonización:** Los 3 productos tienen `"weight": 0`, un volumen genérico o nulo (`volume: 0` / `volume: 1`) y carecen del objeto estándar `dimensions` (`x`, `y`, `z`), además de tener `attributes.location: ""` vacío. Es necesario enviar pesos y dimensiones espaciales reales y consistentes (`dimensions: { "x": largo, "y": ancho, "z": alto }`) para que el motor de *Cartonization* asigne el vehículo correcto (moto vs. van) y el pasillo para optimizar la ruta del picker.
- **5. Validación de Mayoría de Edad en Entrega (*Delivery Validations*):** El ítem `"410064"` es una bebida alcohólica (*Cerveza Lata Clamato Sol*). Se recomienda activar **Delivery Validations por Categoría (+18 años)** y `add_delivery_code: true` para que el conductor verifique la edad del receptor antes de entregar.

4.4. Payload JSON 100% Corregido y Adaptado a TongoMart (Uganda)

A continuación se presenta el payload completo corregido sintáctica, semántica y operativamente para el escenario de TongoMart en Kampala, Uganda (utilizando `"address"` y `"dimensions": { "x", "y", "z" }`):


```
{
  "start": "2026-11-04T18:00:00.000Z",
  "end": "2026-11-05T18:00:00.000Z",
  "slot_size": 60,
  "minimum_slot_size": 30,
  "operational_models_priority": [
    "FULL_SERVICE",
    "PICK_AND_DELIVERY"
  ],
  "store_reference": "1234",
  "currency_code": "UGX",
  "is_big_order": false,
  "payment_info": {
    "order_value": 172.3,
    "method": "CASH"
  },
  "origin": {
    "name": "TongoMart Acacia Mall Darkstore",
    "address": "Plot 14-18 Cooper Road, Kisementi",
    "address_two": "Kololo, Ground Floor Loading Bay",
    "country": "Uganda",
    "city": "Kampala",
    "state": "Central Region",
    "zip_code": "00256",
    "latitude": 0.3406,
    "longitude": 32.5862
  },
  "destination": {
    "name": "Residencia Cliente Nakasero",
    "address": "Plot 24 Nakasero Road",
    "address_two": "Apt 4B, Opposite State House Road",
    "description": "Portón gris, entregar en recepción",
    "country": "Uganda",
    "city": "Kampala",
    "state": "Central Region",
    "zip_code": "00256",
    "latitude": 0.3264,
    "longitude": 32.5789
  },
  "job_items": [
    {
      "id": "394272",
      "name": "Salchicha Viena Chimex",
      "photo_url": "https://cdn.tongomart.ug/catalog/394272.png",
      "unit": "pz",
      "sub_unit": "pz",
      "quantity": 1,
      "sub_quantity": 1,
```

```
      "weight": 0.25,
      "volume": 350,
      "dimensions": {
        "x": 15,
        "y": 8,
        "z": 3
      },
      "price": 21.9,
      "comment": "Revisar fecha de vencimiento amplia",
      "attributes": {
        "location": "Pasillo 4 - Refrigerados",
        "picking_index": "1",
        "category": "Carnes Frías"
      }
    },
    {
      "id": "404670",
      "name": "Tortilla De Harina",
      "photo_url": "https://cdn.tongomart.ug/catalog/404670.png",
      "unit": "pz",
      "sub_unit": "pz",
      "quantity": 1,
      "sub_quantity": 1,
      "weight": 0.36,
      "volume": 480,
      "dimensions": {
        "x": 20,
        "y": 20,
        "z": 1.2
      },
      "price": 25.9,
      "comment": "Empaque sellado sin roturas",
      "attributes": {
        "location": "Pasillo 2 - Panadería",
        "picking_index": "1",
        "category": "Panadería y Tortillas"
      }
    },
    {
      "id": "410064",
      "name": "Cerveza Lata Clamato Sol",
      "photo_url": "https://cdn.tongomart.ug/catalog/410064.png",
      "unit": "pz",
      "sub_unit": "pz",
      "quantity": 5,
      "sub_quantity": 5,
      "weight": 0.473,
      "volume": 520,
      "dimensions": {
        "x": 6.6,
        "y": 6.6,
        "z": 15.7
      },
      "price": 24.9,
      "comment": "Producto sujeto a verificación de mayoría de edad (+18)",
      "attributes": {
        "location": "Pasillo 8 - Licores y Bebidas",
        "picking_index": "2",
        "category": "Bebidas Alcohólicas"
      }
    }
  ]
}
```

PUNTO 5 DEL CASO PRÁCTICO

5. Propuestas Técnicas de Mejora para la API

Con base en la experiencia de diseño e integración del caso TongoMart, se formulan **5 propuestas constructivas de ingeniería de software** orientadas a optimizar la experiencia del desarrollador (*Developer Experience - DX*), reducir la fricción de integración y fortalecer la resiliencia operativa de **La Plataforma Logística**:

Propuesta 1: Unificación del Esquema `payment_info` entre Endpoints

- **Problema Actual:** En `POST /jobs/availability/v2`, el objeto `payment_info` tiene una estructura plana (`{ "order_value": 172.3, "method": "CASH" }` con `currency_code` en la raíz), mientras que en `POST /jobs` y `PUT /jobs/{jobId}/payment_info` cambia a un esquema anidado (`{ "currency_code": "UGX", "prices": { ... }, "payment": { ... }, "invoice": { ... } }`).
- **Solución Propuesta:** Adoptar un único contrato canónico para `payment_info` en todos los endpoints del ciclo de vida del trabajo (permitiendo que sub-objetos como `invoice` sean opcionales durante la consulta de disponibilidad).
- **Impacto:** Permite reutilizar el mismo DTO/Serializador en el Middleware de Magento y reduce errores de transformación entre pasos.

Propuesta 2: Tipado Numérico o Polimórfico en `attributes.picking_index` y Consistencia de Nombres

- **Problema Actual:** El campo `job_items[].attributes.picking_index` representa una secuencia numérica de recorrido de pasillos (`1` , `2` , `3`), pero el contrato exige que se envíe como `string` (`"1"`), rechazando enteros con error `422` . Asimismo, en `POST /jobs` coexisten llaves como `client_reference` junto a `client_Reference_number` (con `R` mayúscula).
- **Solución Propuesta:** Aceptar en el JSON Schema `oneOf: [{ "type": "string" }, { "type": "integer" }]` tanto para `picking_index` como para `store_reference` (normalizándolos internamente en el API Gateway) y estandarizar todas las propiedades a estricto `snake_case` .
- **Impacto:** Elimina rechazos de validación por diferencias semánticas menores durante las pruebas de certificación.

Propuesta 3: Soporte Nativo de Cabecera `Idempotency-Key` en `POST /jobs`

- **Problema Actual:** En integraciones con Magento, un *timeout* de red al invocar `POST /jobs` puede disparar un reintento automático desde el middleware del comercio, corriendo el riesgo de duplicar la orden física en tienda si la primera petición sí había alcanzado a procesarse.
- **Solución Propuesta:** Soportar el header estándar `Idempotency-Key: <UUID-v4>` en `POST /jobs` , almacenando en caché (TTL 24h) la respuesta asociada a la llave para retornar el mismo `job_id` ante reintentos idénticos sin duplicar el trabajo.
- **Impacto:** Garantiza procesamiento *exactly-once* en la creación de órdenes y protege la capacidad de slots.

Propuesta 4: Respuestas de Error Estructuradas bajo el Estándar RFC 7807 (*Problem Details*)

- **Problema Actual:** Cuando un payload complejo presenta múltiples errores simultáneos (como ocurrió en el JSON de TongoMart), un mensaje genérico `400 Bad Request - Missing attributes` dificulta identificar exactamente qué rutas del JSON fallaron.
- **Solución Propuesta:** Estandarizar las respuestas `400` y `422` bajo **RFC 7807 / RFC 9457 (`application/problem+json`)** indicando el `pointer` exacto del atributo, el código de error y el tipo esperado:

```
{
  "type": "https://api.logistics-platform.io/errors/validation-error",
  "title": "Unprocessable Entity - Payload Validation Failed",
  "status": 422,
  "detail": "The request body contains 4 validation errors.",
  "instance": "/jobs/availability/v2",
  "errors": [
    {
      "pointer": "/slot_size",
      "code": "MISSING_REQUIRED_FIELD",
      "message": "Field 'slot_size' is required and must be a multiple of 15."
```

```
    },
    {
      "pointer": "/end",
      "code": "INVALID_DATE_RANGE",
      "message": "'end' (2026-01-28T01:55:36.048Z) must be greater than 'start' (2026-01-30T17:52:29.991Z)."
    },
    {
      "pointer": "/store_reference",
      "code": "INVALID_TYPE",
      "expected_type": "string",
      "received_type": "number"
    },
    {
      "pointer": "/job_items/1/quantity",
      "code": "VALUE_OUT_OF_RANGE",
      "message": "Item quantity must be greater than 0."
    }
  ]
}
```

Propuesta 5: Exposición de Cabeceras Estándar de Rate Limit (`X-RateLimit-*`)

- **Problema Actual:** El endpoint `/jobs/availability` aplica un límite de **400 peticiones por minuto por API Key**. Sin cabeceras de telemetría en las respuestas, el middleware de Magento no puede anticipar qué tan cerca está de agotar su cuota durante eventos de alto tráfico.
- **Solución Propuesta:** Incluir en cada respuesta las cabeceras estándar `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` y `Retry-After`.
- **Impacto:** Permite al Middleware de TongoMart activar colas o caché preventiva de slots en Kampala antes de recibir respuestas `429 Too Many Requests`, protegiendo la tasa de conversión del checkout.